

Computational Fluid Dynamics II

Lab 3 : Breaking of a model dam

Due date: 04 August 2025

Deliverables: Group report + Video

Report format: Typed

Summer Semester 2025

Members (alphabetical):

1. Hemanth Siva Kumar Ravanam - 23423928
2. Mihir Ajit Lonkar - 23515994
3. Mohan Kumar Meesala - 23423951

Task 1

Breaking of a dam: Important Points

1. This simulation uses `interFoam`, which is based on the volume of fluid (VOF) method in which a specie transport equation is used to determine the relative volume fraction of the two phases, or phase fraction α , in each computational cell.
2. It uses the VOF (Volume of fluid) method, where a phase transport equation is solved for the `alpha` (volume fraction) field. Physical properties are calculated as weighted averages based on `alpha`.
3. During the collapse, the water impacts an obstacle at the bottom of the tank and creates a complicated flow structure, including several captured pockets of air
4. The case directory 'damBreak' is copied to our run directory
5. The mesh is generated using the `blockMesh` utility and is represented by 5 blocks
6. Patches are identified as : `leftWall`, `rightWall`, `lowerWall`, `atmosphere` and `defaultFaces`
7. `defaultFaces` is empty for 2D cases, indicating that they are not solved.
8. Walls (`leftWall`, `rightWall`, `lowerWall`) are identified as impenetrable walls.
9. Surface tension effects at walls are ignored by applying `zeroGradient` to the `alpha` field on walls, instead of `alphaContactAngle`.
10. The top boundary is free to the atmosphere so needs to permit both outflow and inflow according to the internal flow.
11. `totalPressure`, for pressure (fixed value from total pressure and local velocity).
12. `pressureInletOutletVelocity`, for velocity (zero gradient, fixed tangential for inflow).
13. `inletOutlet`, for other fields (zero gradient for outflow, fixed value for inflow).
14. At all wall boundaries, the `fixedFluxPressure` boundary condition is applied to the pressure field, which calculates the normal gradient from the local density gradient.
15. Non-uniform initial condition for `alpha` is set using the `setFields` utility and `setFieldsDict`.
16. `setFieldsDict` uses `defaultFieldValues` (e.g., `alpha.water 0`) and regions with `boxToCell` to define the water column (`alpha.water 1`).
17. It is recommended that a backup is made before `setFields` is executed. In the *damBreak* tutorial, the `alpha.water` field is initially stored as a backup only, named `alpha.water.orig`.
18. Phases (water, air) and surface tension (`sigma`) are specified in `phaseProperties`. The phase fraction equation is solved for `alpha.water`.
19. Kinematic viscosity (`nu`) and density (`rho`) for each fluid are set in `physicalProperties.air` and `physicalProperties.water` using a constant viscosity model.
20. Gravitational acceleration (`g`) is uniform and specified in the `g` file.
21. Turbulence modeling is off (simulationType `laminar`).
22. By specifying `adjustTimeStep` to be yes and the maximum *Co* for the phase fields, `maxAlphaCo`, and other fields, `maxCo`, to be 0.5. The upper limit on time step `maxDeltaT` can be set to a value that will not be exceeded in this simulation.
23. The `interFoam` solver uses the multidimensional universal limiter for explicit solution (MULES) method, created by OpenCFD, to maintain boundedness of the phase fraction independent of underlying numerical scheme, mesh structure, etc.
24. `div(rhoPhi,U)` (momentum convection) uses Gauss linearUpwind `grad(U)` for accuracy.
25. `div(phi,alpha)` uses the vanLeer scheme and `div(phiRb,alpha)` can similarly use the vanLeer scheme, but generally produces smoother interfaces using the linear scheme.
26. The `cAlpha` keyword is a factor that controls the compression of the interface where: 0 corresponds to no compression; 1 corresponds to conservative compression; and, anything larger than 1, relates to enhanced compression of the interface.
27. The simulation is run using the command: `interFoam`.
28. Results can be post-processed to observe the development of the `alpha.water` phase fraction over time.

Task 2 and 3

Here we have created family of meshes to check the grid convergence and resolution of the result or simulation. The meshes which we have created are coarsest, medium and finest meshes which have control volumes of 37300(**Mesh 1**) for coarsest, 74264(**Mesh 2**) for the medium and 149200(**Mesh 3**) for the finest meshes.

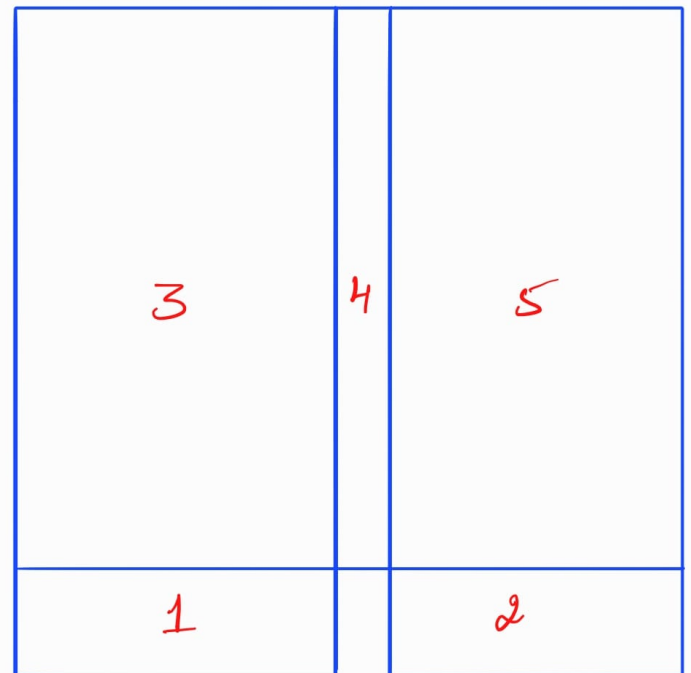
For generating meshes we have used the default blockMesh directory which is available in the tutorials and made changes in the hex block where we have increased the number of grid points and also changed gradients for increasing resolution at particular regions which is near wall region on both left and right side as well as the bottom surface.

We have decided to increase the resolution at the near wall regions because when water falls freely due to gravitational force it encounters a small block at the middle of bottom surface and splashes on the right wall and again comes back and splashes on the left wall until its going to be settled down till there is no disturbance. This is our initial guess and we have run the default case directory and have seen the result how water is going to be change its position with respect to time and what we have guessed is correct.

So, we have increased the grid resolution on the left and right side of the wall and also at the bottom surface, here we are simulating multiphase, to be exact two phase namely air and water where we encounter the air water mixture at some particular time period where we can view water bubbles inside the water for a period of time then air bubbles comes to the surface as we know because of the difference in density of both fluids.

For building there meshes we have chosen the default blockMesh in which we have 5 hexblocks

In the all the three meshes we have to match number of nodes in x direction in block 1 and 3 as well as 2 and 5, also we have to match number of nodes in y direction in block 3, 4 and 5. By doing so we won't have any problem while generating mesh for doing simulation if not there will be unexpected or unknown errors due to mismatch in number of nodal points and mesh will not be generated and also we have given gradient in y direction in all the blocks and given gradient in x direction in 1, 2, 3 and 5 blocks block 1 and 3 gradient should match in x direction, block 2 and 5 gradient should match in x direction, similarly block 1 and 2 gradient should match in y direction and also block 3, 4 and 5 gradient should match in y direction, in this way mesh looks more structured and uniform and this is what we are aiming to achieve in all three meshes.



Task 4 and 5

With default fvSchemes we have ran simulation all three meshes and also, we have taken finest mesh of all and changed time integration scheme and each of the divergence scheme and ran the simulation to compare it with the default fvSchemes.

The **default fvSchemes** for time integration is Euler and divergence schemes is Gauss linear.

I have taken two cases,

Case1: Time integration is CrankNicolson.

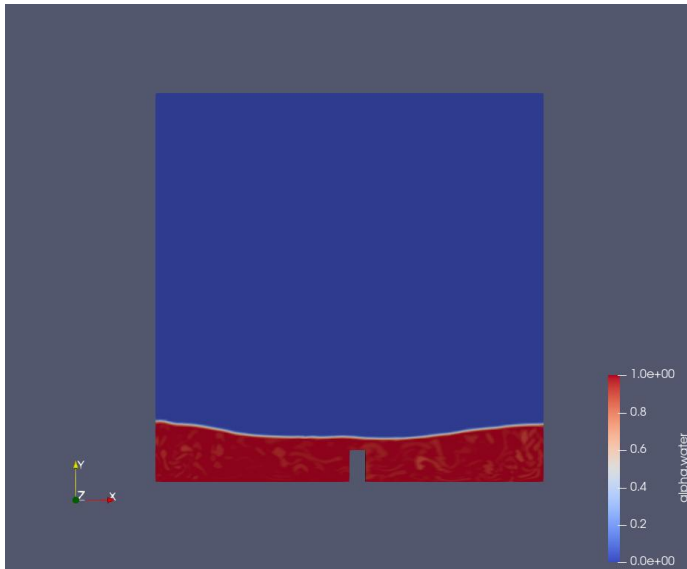
Divergence scheme is QUICK.

Case2: Time integration is Euler.

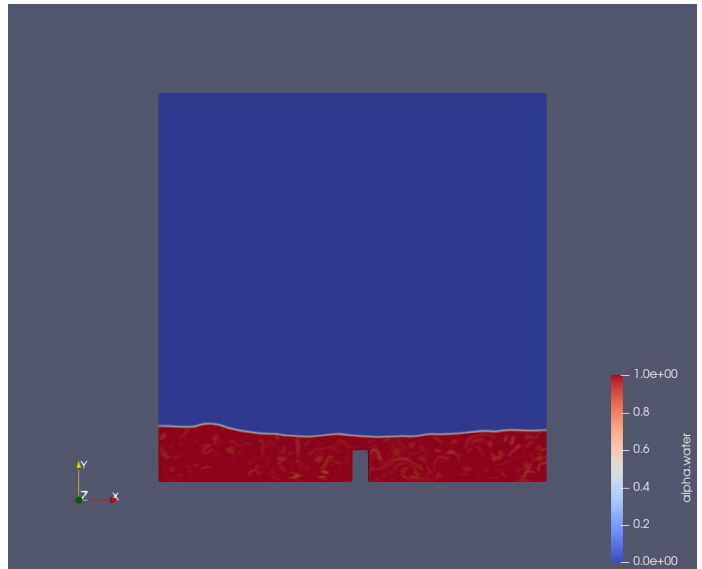
Divergence scheme is QUICK.

For time integration we are only being able to take either Euler or CrankNicolson, open Foam itself giving an error prompt that's this particular case directory only uses Euler or CN, when I have tried to use backward scheme in time integration.

Comparison of default fvSchemes after 10 seconds of run time on all three meshes.



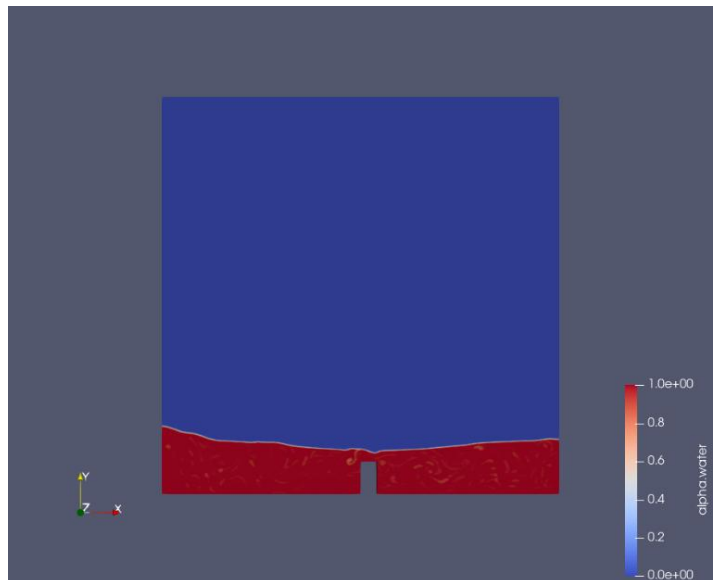
Coarse Mesh



Medium Mesh

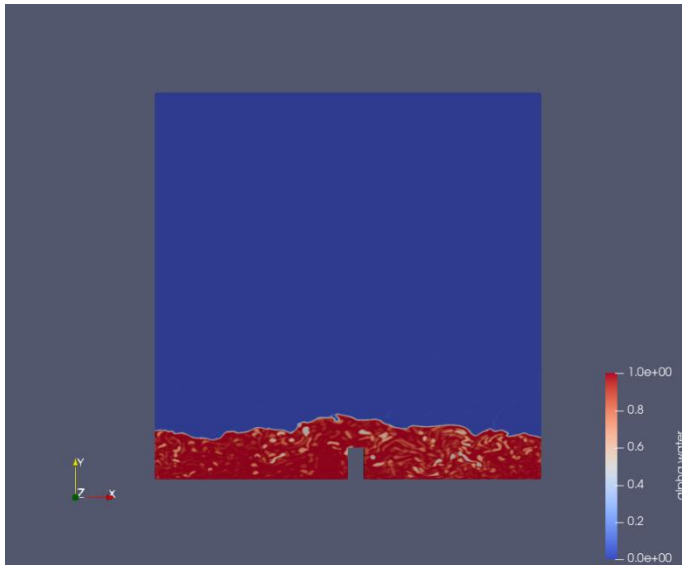
Here we can observe that there is clear difference in all three grid resolutions, particularly at the top surface or layer of water where air and water separates, we can observe sharp difference between two fluids and also, we can observe that even after 10 seconds there is still some air water mixture which indicates the water is still in motion.

The more we refine the mesh, the smaller the scales become and we can observe the difference in the results as well.

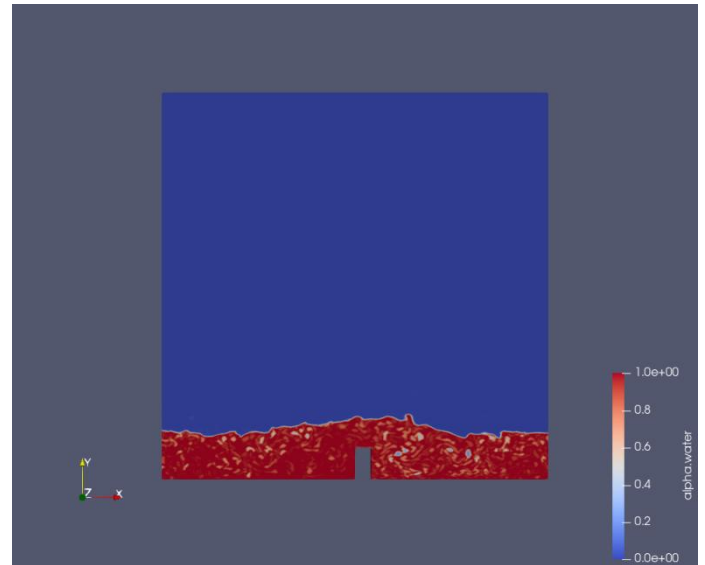


Fine Mesh

Comparison of same grid(coarse) with different fvSchemes after 3 seconds of simulation.

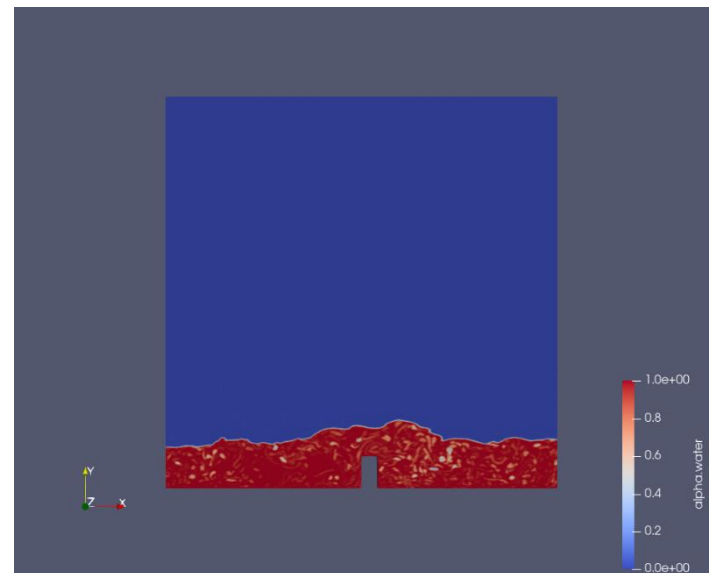


Default fvSchemes / case



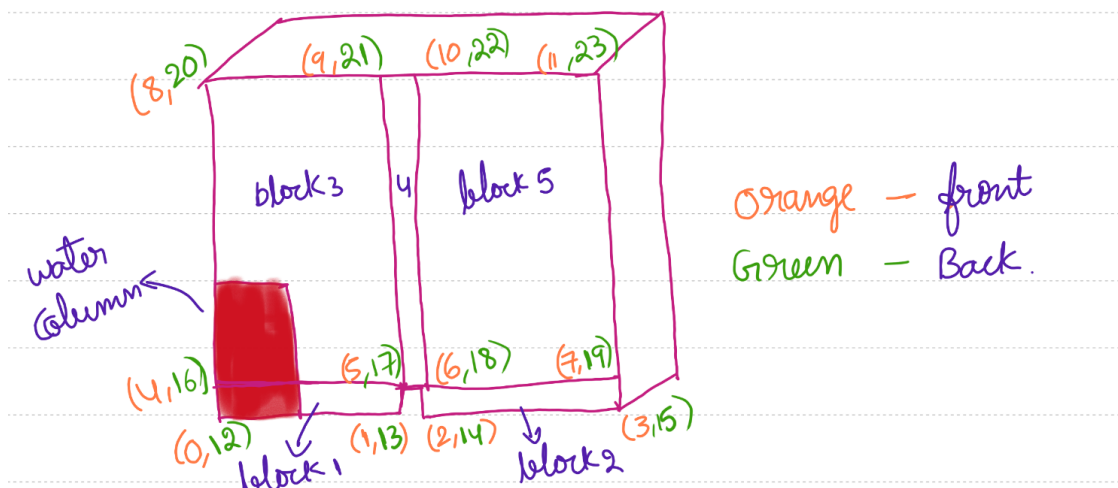
Case1

Here we can observe that in all three cases the water shape looks different and in the case1 where we have used CrankNicolson in time integration and QUICK in divergence schemes has the most air water mix visibility than other two because Crank Nicolson is second order accurate and QUICK is third order accurate, where Euler is first order accurate and Gauss Linear upwind is second order accurate but CrankNicolson is unconditionally stable. Due to this reason case1 has clear difference in the result of air water mixture. And case 2 has the second most accuracy in result where in default case we can observe that air water mixture is not as sharp as the other two cases, in this case we have used Euler in time integration which is 1st order and used Gauss liner upwind in divergence schemes which is 2nd order. Here air bubble inside the water is trying to diffuse in the water which leads to slight divergence in the result not being most accurate out of all cases we have considered.



Case2

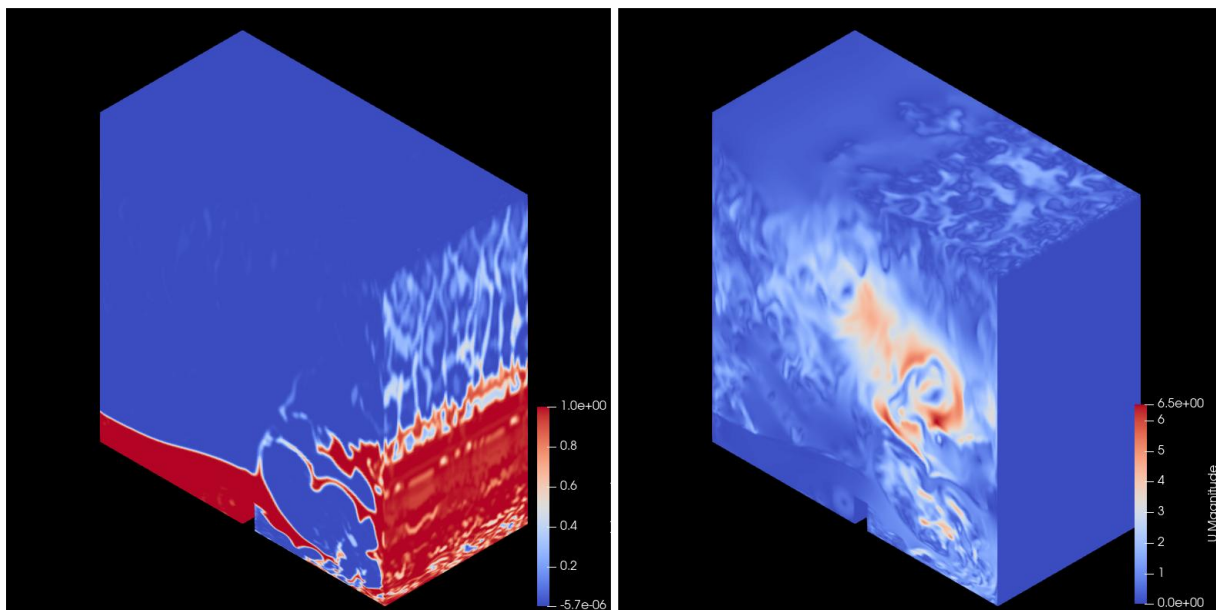
Task 6



By replacing the vertices in the z-direction with 2 instead of 0.1, as scale is multiplied by 0.146, which gives our desired length 0.292m. We can easily figure out the front and back vertices by just analysing the blocks in the blockDict, which is clearly shown in the image attached above. The faces are then given periodic boundary conditions and are extruded with 50 layers in the spanwise direction. The reason for choosing 50 layers is that the length of our domain in 2D is 0.582m in all directions, which contains around 80-120 cells in x direction which results in 37.3k cells for our coarsest mesh, meanwhile the length in spanwise is only half of that so the spanwise length is divided into 50 layers. The resulted 3D mesh has 1.8 million cells, which is below the threshold value given. This also saves computational cost and time by choosing the approximate layers in spanwise direction.

Task 7

The 3D simulation was run in parallel on 20 cores, with same solver settings as the 2D setup. But with front and back cyclic boundary conditions. Image: alpha wate (left) and Velocity magnitude (right).



We can see that the 3D simulation has scales in the Z direction too, which confirms correctness of implementation. The velocity is 0 on the walls (right image) which is as expected. In the Z direction, the flow is not symmetric, thus concluding that this multiphase flow is a multidirectional phenomenon.

Task 8

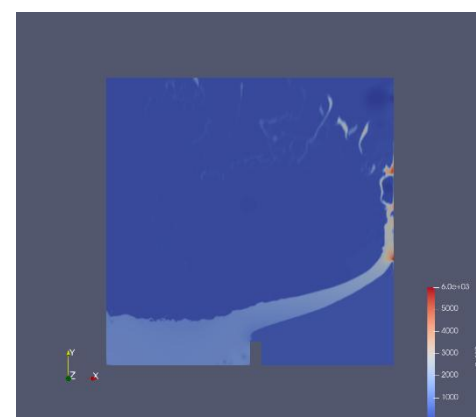
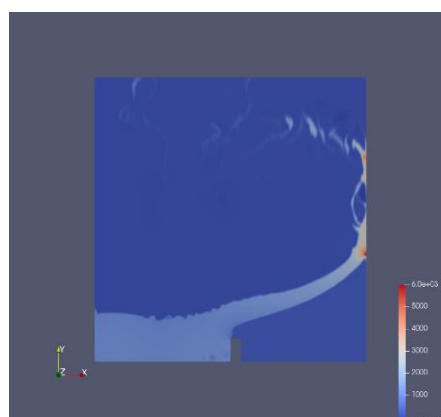
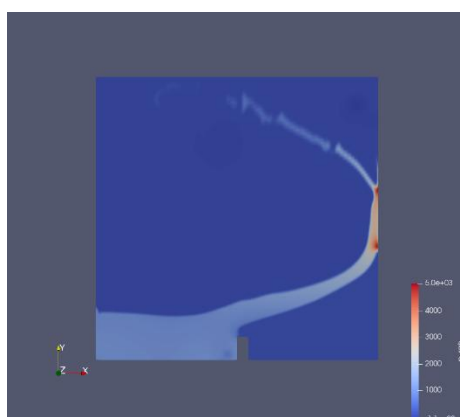
The animation was created using snapshots of the flow. To create a real time simulation, 25 snapshots or time steps were taken each second. This was done (writeTime 0.040). This gives us a video with 25 frames per second. Identification of the walls is done using the fact that walls have no velocity and hence this can be used to identify them. The surface of the water is identified using alpha value (0.5). The images were converted to video using a simple python script.

Task 9

Mesh 1

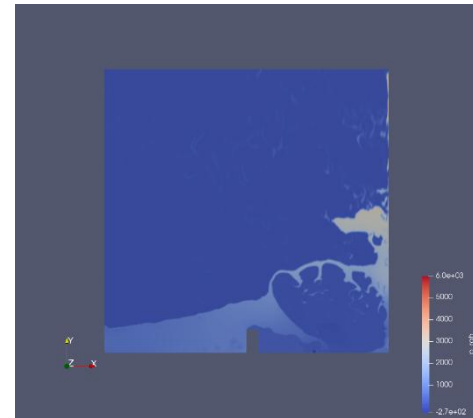
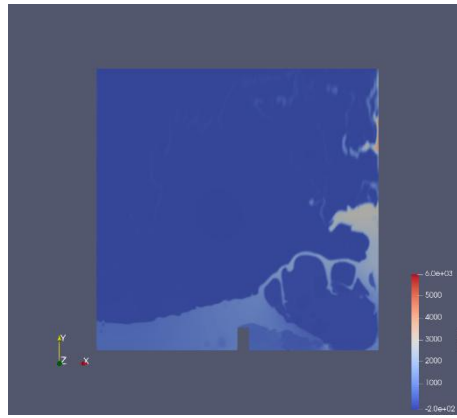
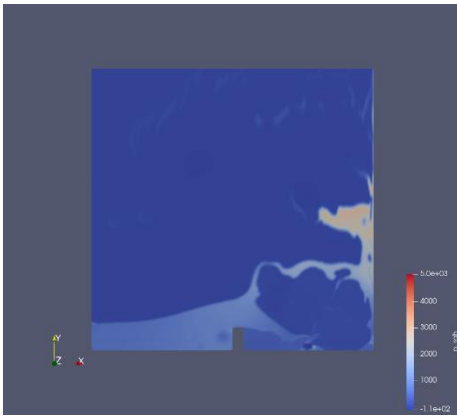
Mesh 2

Mesh3

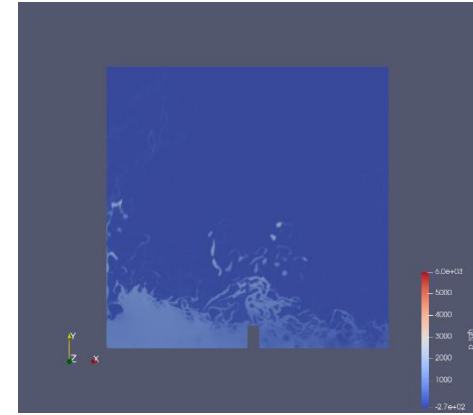
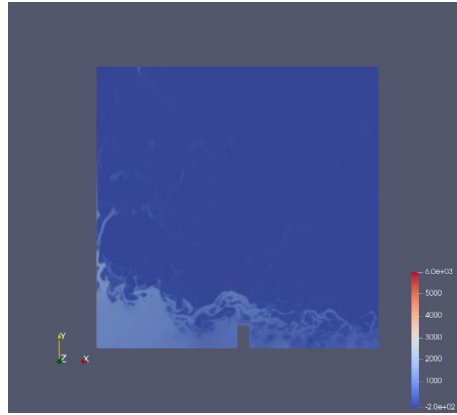
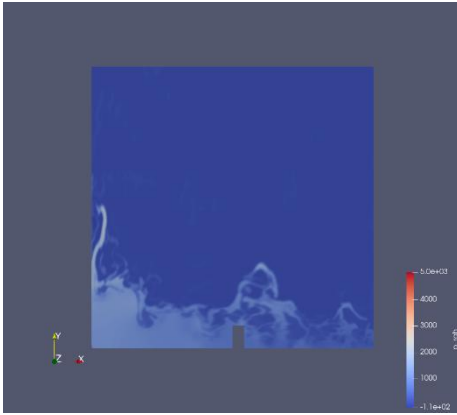


At time 0.35s

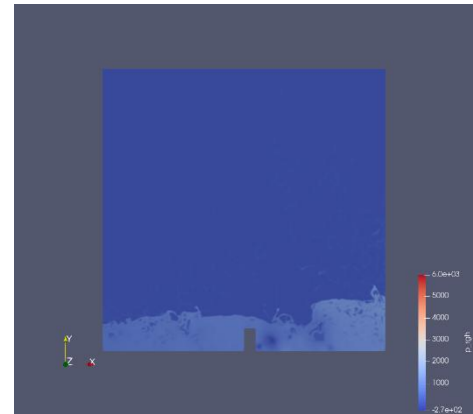
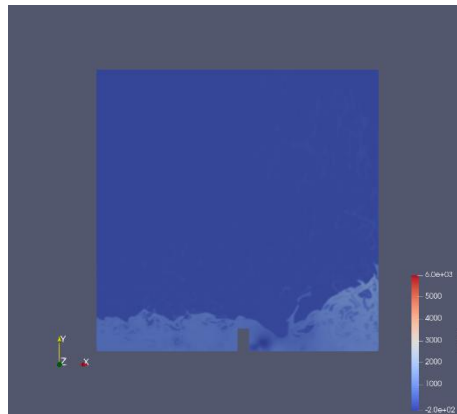
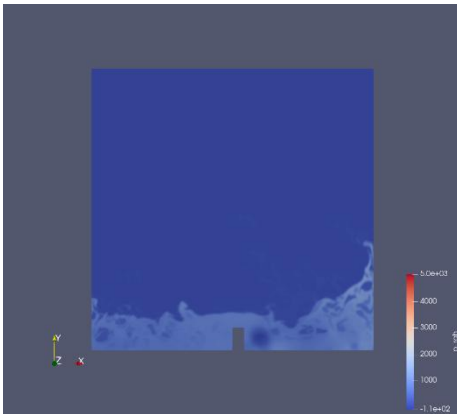
At time 0.55s



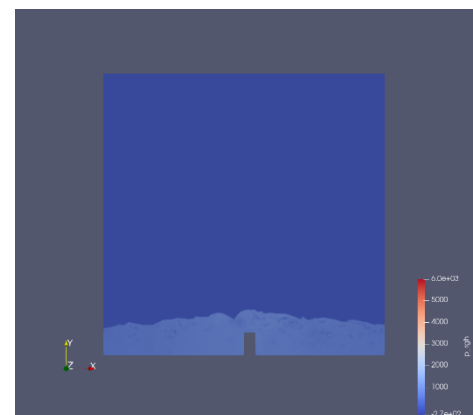
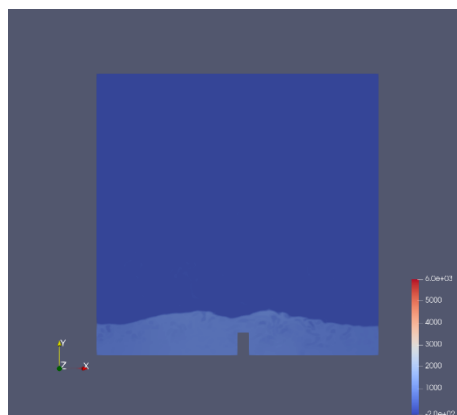
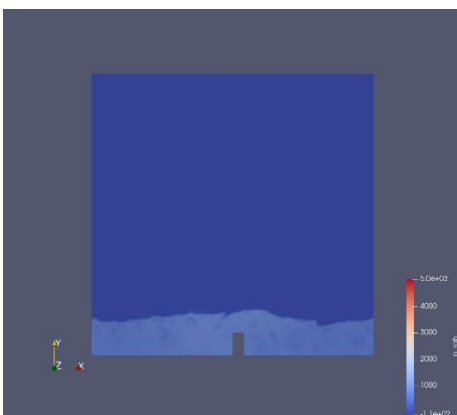
At time 1s



At time 1.75s



At time 3.65s



The plots are given for p_rgh which is the difference between total pressure and hydrostatic pressure($p-p*g*h$) which is important for buoyant and multiphase cases, which avoids numerical problems when dealing with large hydrostatic pressure gradients and also improves stability and accuracy in the solver. The important instances are chosen based on where the physics or flow properties changes abruptly like hitting the wall, rebounding from right wall, again rebounding and stabilizing of the fluids. For the meshes, the flow patterns and the values that they attain at the walls are almost similar to each other, so by considering this, we can state that the grid convergence .

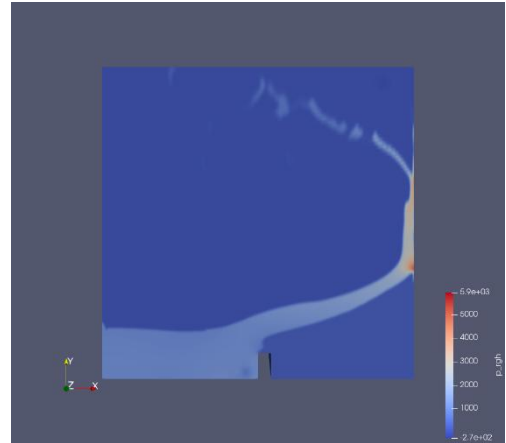
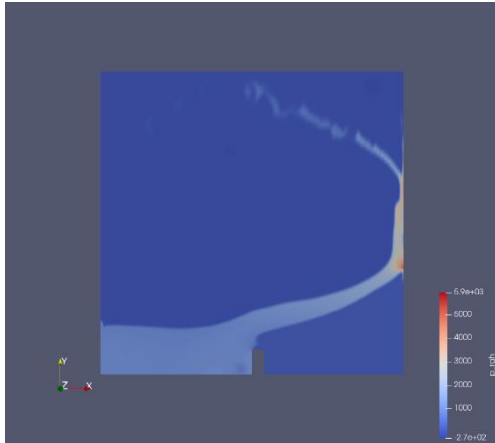
ref(<https://www.openfoam.com/documentation/guides/latest/doc/guide-applications-solvers-variable-transform-p-rgh.html>)

Task 10

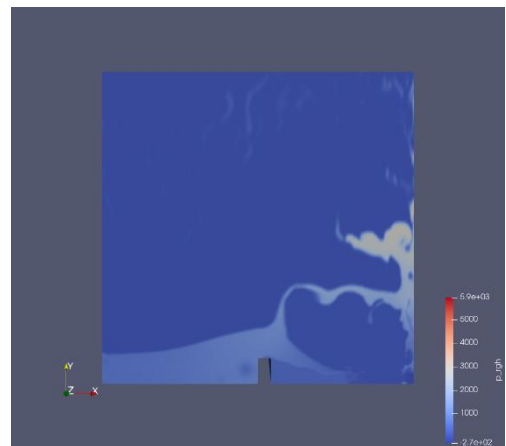
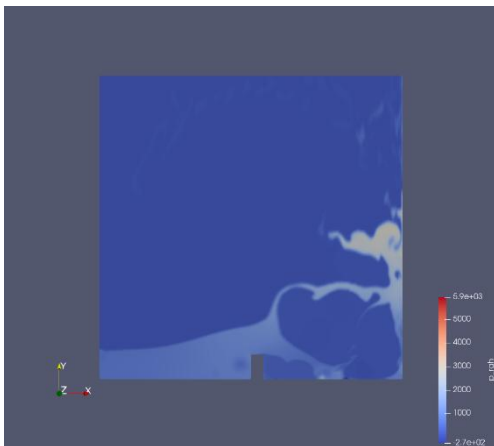
2D

3D

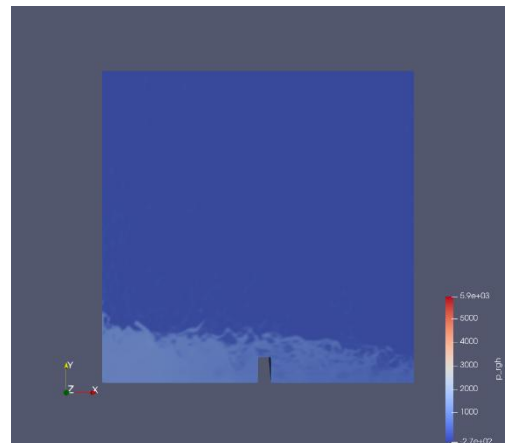
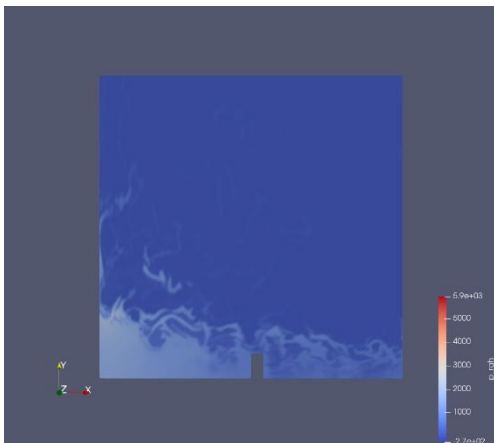
At time 0.35s



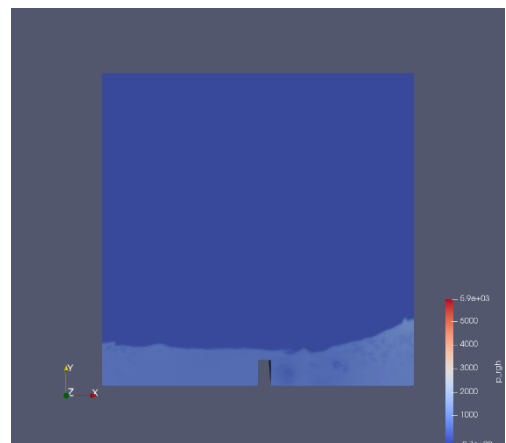
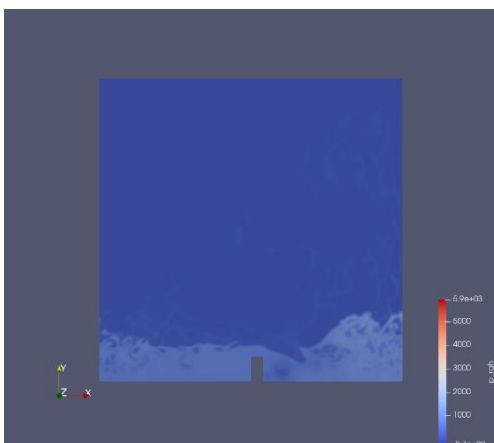
At time 0.55s



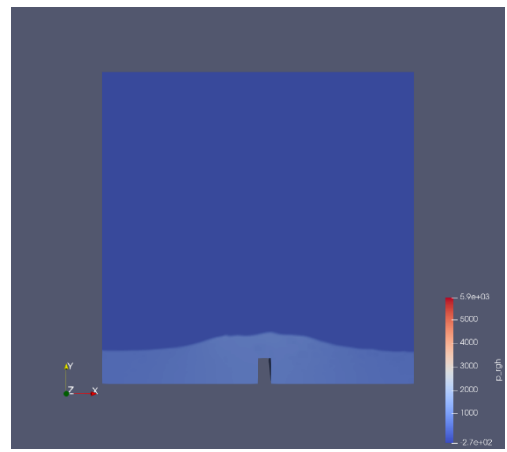
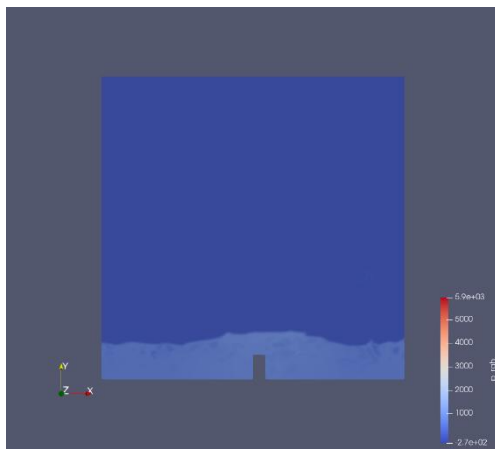
At time 1s



At time 1.75s



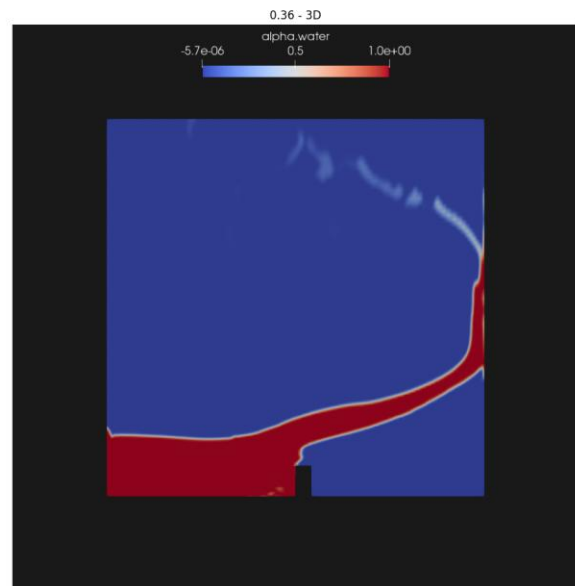
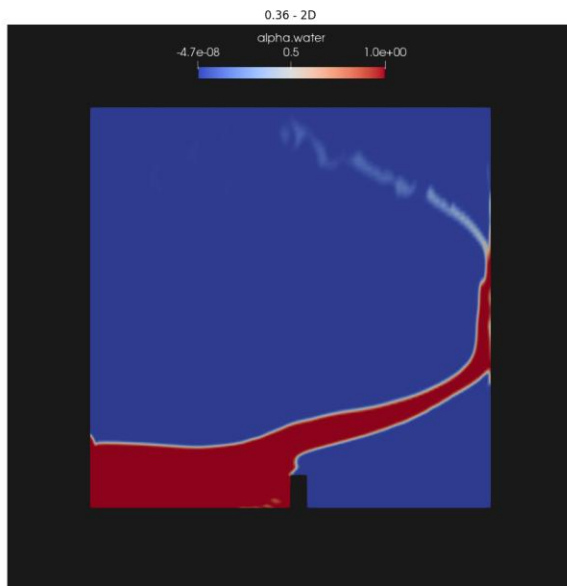
At time 3.65s



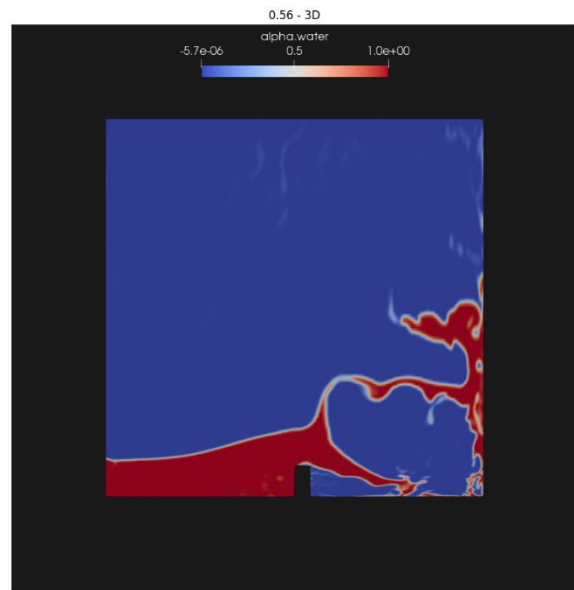
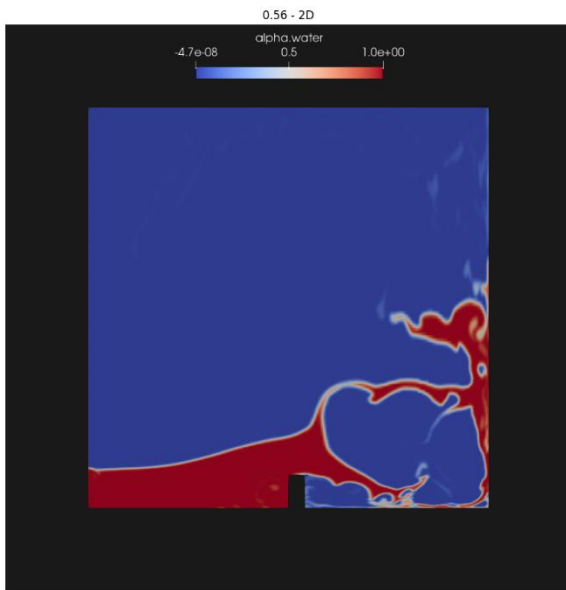
We can observe some changes in 2D and 3D even though there are same number of cells(37.3k) along the spanwise-direction, this is mainly because the interface location is determined by alpha (0 or 1) and this leads to different interface predictions. The Continuum Surface Force(CSF) model, which depends on $\text{grad}|\alpha|$, behaves differently in 2D and 3D meshes. As CSF acts primarily near the surface, so different surface configurations result in varying force distributions.

Task 11

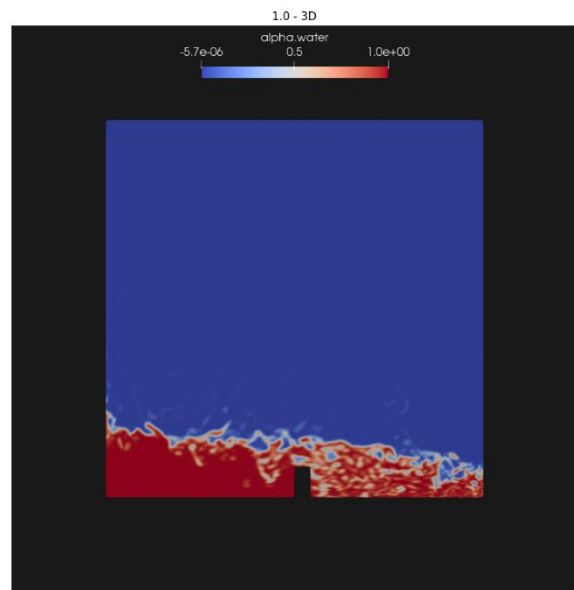
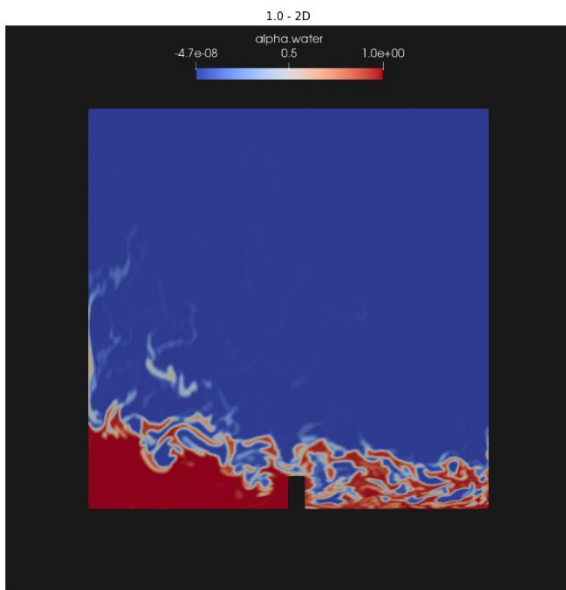
This simulation uses VOF approach which is an interface capturing method. The presence of interface is decided based on the value of alpha. A value between 0 and 1 signifies presence of interface. We know that an additional alpha equation is solved which gives different results in 3D as the interface is decided based on alpha (Slide 36 Lecture 10). Therefore we get different interface eventhough the mesh is the same, but 3D. The CSF is based on $\text{grad}|\alpha|$ which again is different for 2D vs 3D mesh (Slide 37 Lecture 10). CSF acts only near the surface and hence for a different surface values, this quantity differs. Spurious currents are generated in case of 3D simulations due to CSF model. However in this case, it was not observed (Slide 40 Lecture 10).



(Time = 0.36) The first wave splashes on the right wall. We can see that there is no visible difference between the 2D and 3D simulations for the coarsest mesh.

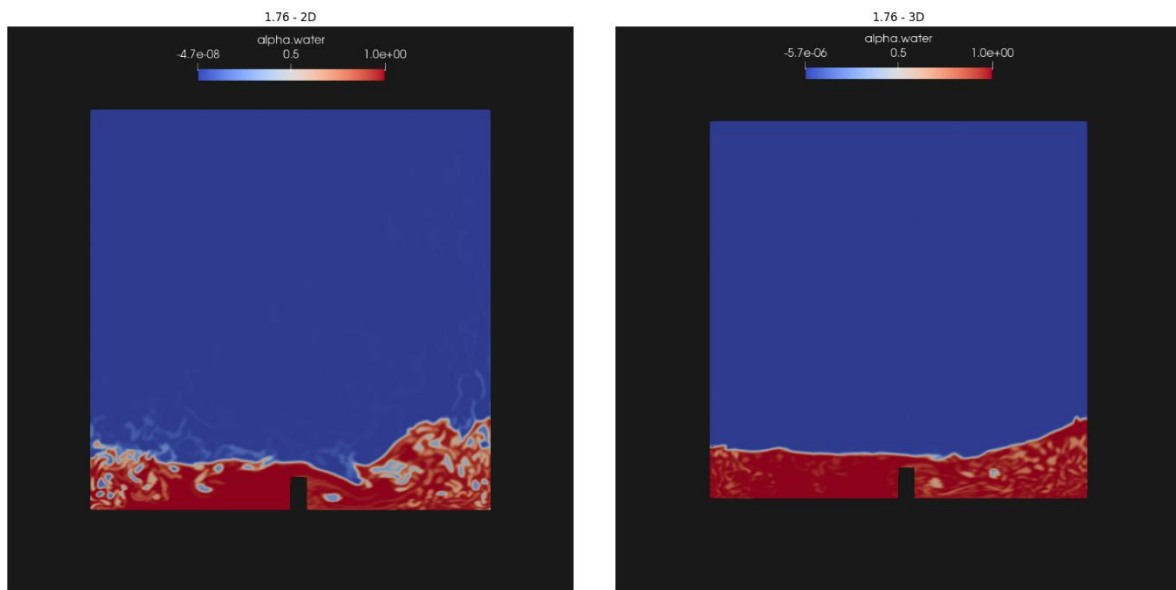


(Time = 0.56) After the wave rebounds from the wall, we start seeing the difference, although they both have the same resolution in the XY plane.

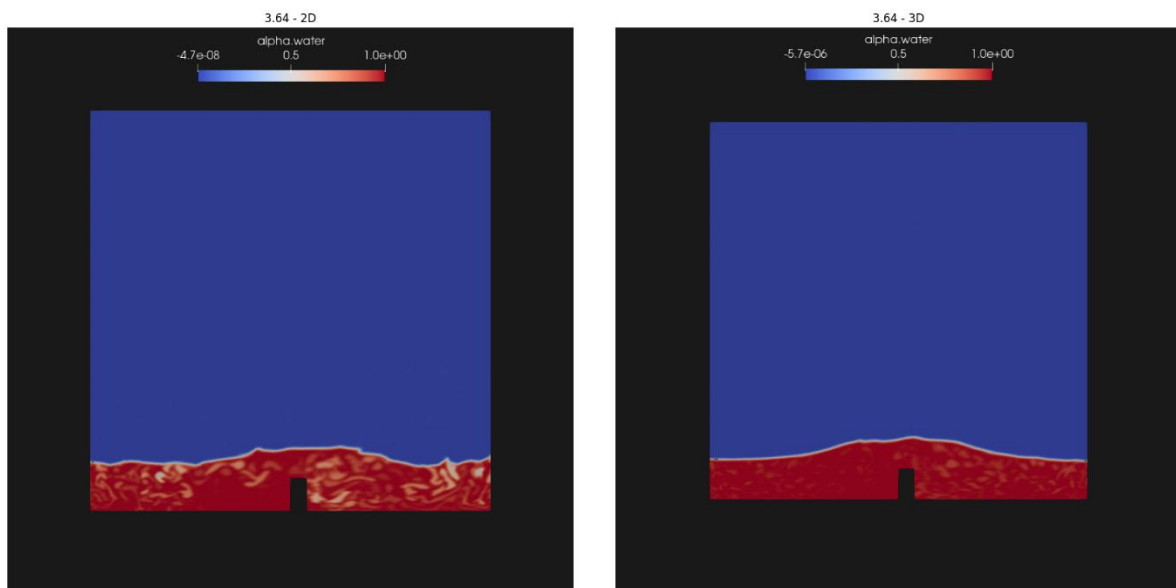


(Time = 1) The splashed wave rebounds from the left wall. We observe that the size of the waves is different in both the cases. The 3D waves seem to have finer scales even though both have same spatial resolution in

XY plane. This may be a result of CSF contribution.



(Time = 1.76) The unsteady waves have now settled a bit and we see that there are differences in 2D and 3D simulation. We observe that for 3D, the result looks more realistic as the interface value alpha remains 1 for most scales (Due to the solver checking the interface in Z direction too).



(Time = 3.64) We continue to see the difference in the size of waves although the overall pattern of wave structure is preserved. The 3D result looks more plausible and physically correct.

Contributions:

Hemanth Siva Kumar Ravanam – 2, 3, 4, 5

Mihir Ajit Lonkar – 7, 8, 11

Mohan Kumar Meesala – 1, 6, 9, 10